CPT108 Assignment 1:
Complexity Analysis

Felix Lianto (2362727)

Instructor: Ho Lam

September 18, 2025

1 Introduction

The *computational complexity* of an algorithm is defined as the minimum amount of resources needed for an algorithm to solve a computational problem [6]. The study behind this is called the *computational complexity theory*. In this area of study, it is most often associated with optimizing the running time and memory space of an algorithm to maximize its efficiency. This report will explore these topics through a basic analysis of common sorting algorithms, which are,

- Bubble Sort,
- Insertion Sort,
- Selection Sort,
- Merge Sort,
- Quick Sort, and
- Heap Sort.

This experiment aims to explore:

1. How each sorting algorithms performs with respect to its execution time,
2. The runtime of each sorting algorithms under certain inputs, and
3. The memory usage of the sorting algorithms, measured through finding the maximum array size can the Java Virtual Machine (JVM) heap allocate for each sorting algorithm under limited heap size.

1.1 Brief Overview of Computational Complexity Analysis

As mentioned previously, computational complexity can be measured in two dimensions, running time and memory space. Running time of an algorithm, more commonly known as the *time complexity*, is equivalent to the amount of "steps" an algorithm needs to take in order to arrive at an answer [6]. In similar fashion, *space complexity* concerns itself with the amount of memory usage an algorithm requires in order to run its code. Both types of resources are measured with *asymptotic notation*, where the amount of "steps" required is represented by a function of its input length [1]. This is to ensure that its performance is language and machine independent, that is, the algorithm will perform similarly when dealing with larger and larger inputs, regardless of the programming language its implemented on, or the machine its working on.

1.2 Asymptotic Analysis

As stated before, algorithms can be measured through this universal language of asymptotic notation. These algorithms are commonly grouped together

to form *complexity classes*, where given a large enough input, the performance will grow similarly to each other. Figure 1 shows the common orders of complexity that frequently occurs in asymptotic analysis.

$$O(1) < O(\log n) < O(n) < O(n^c) < O(c^n) < O(n!)$$

Constant < Logarithmic < Linear < Polynomial < Exponential < Factorial

Figure 1: Common complexities of asymptotic functions

It is important to emphasize that the asymptotic complexity, regardless if it is time or space, is only a loose estimate of the actual amount of steps required for the algorithm to return successfully. Indeed, some algorithms, even if they share the same complexity class, can still perform better or worse than other algorithms.

To accurately analyze an algorithms' performance, asymptotic notation oftentimes deal with extremes, that is, analyzing the lower and upper bound of a function. These bounds are referred to as the *Big-Oh notation* and the *Big-Omega notation*. Formally, they are defined as follows.

Defintion 1 (Big-Oh Notation [4])**.** *$f(n)$ is a lower bound of $g(n)$ if there exists some constant c and n_0 such that for every $n \geq n_0$, $f(n) \leq c \cdot g(n)$. This can be commonly written as $f(n) = O(g(n))$.*

Defintion 2 (Big-Omega Notation [4])**.** *$f(n)$ is an upper bound of $g(n)$ if there exists some constant c and n_0 such that for every $n \geq n_0$, $f(n) \geq c \cdot g(n)$. This can be commonly written as $f(n) = \Omega(g(n))$.*

One more important asymptotic notation is the *Big-Theta notation*, which is the intersection of both the Big-Oh and Big-Omega notation, that is, both of the upper and lower bounds of a function can be defined by the same function with different constants. Formally, this is stated as follows.

Defintion 3 (Big-Theta Notation [4])**.** *$f(n)$ is a nice, tight bound of $g(n)$ if there exists some constant c_1, c_2 and n_0 such that for every $n \geq n_0$, $f(n) \leq c_0 \cdot g(n)$ and $f(n) \geq c_1 \cdot g(n)$. This can be expressed as $f(n) = \Theta(g(n))$.*

1.3 Worst-case, Best-case, and Average-case Time Complexities

When discussing the efficiency of algorithms with respect to the runtime, it is without a doubt that the type of input can influence the performance of the algorithms. To account for this, the time complexity of these algorithms are split according to its input type, referred to as *worst-case*, *best-case*, and *average-case*. The worst-case time complexity is the maximal function of the number of steps taken for any input of size n [4]. In contrast, the best-case

time complexity is the minimal function given any input of size n [4]. Finally, between the two extremes, the average-case time complexity, or the *expected time*, is the average function of the number of steps across every input of size n [4].

One important to note is that these cases only deal with the input supplied to the algorithm. Oftentimes, these functions would contain unnecessary details about the amount of steps, in which case the asymptotic notation is utilized to describe a loose estimate of the number of steps required. This also implies that for each case, there would be a lower and upper bound that can be specified by asymptotic notation.

1.4 Time and Space Complexity of Sorting Algorithms

The focus of this report paper is to investigate the time and space complexity of different sorting algorithms. Thus, before presenting the results of the experiment, a discussion about these sorting algorithms and their theoretical performance will be held in this section. This section will also serve as the hypothesis towards the research questions posed above.

In order to keep this subsection brief, a summary of the time complexities for each case and memory complexity for the sorting algorithms tested will be provided in Table 1.

[redacted]	Time Complexity			[redacted]
Name of Algorithm	Worst-case	Average-case	Best-case	Memory Complexity
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Table 1: Summary of the performance of sorting algorithms [2]

From this table, it is expected that for some random integer array input, the performance would follow the average-case column. That is, Merge Sort, Quick Sort, and Heap Sort would outperform Bubble Sort, Insertion Sort, and Selection Sort. For the worst-case and best-case columns, it will depend on what inputs are inserted into each function. However, differences in time performance between Selection Sort, Merge Sort, and Heap Sort, should not exist, as both their worst-case and best-case time complexity are equal. In other words, these algorithms perform independent of the data supplied.

For Bubble Sort and Insertion Sort, it is expected that their best-cases would be inputs already sorted in ascending order, whilst their worst-cases are inputs sorted in descending order. Furthermore, Merge Sort should see its worst-case performance when either the input is sorted in ascending or descending order.

This happens because the algorithm will pick the smallest and biggest pivot every time, thus the list division will always be unequal. Thus, the function will recurse on the same left and right lists twice, thus the $O(n^2)$ time complexity.

Finally, based on the memory complexity column, it is expected that Merge Sort will only sort a small sized integer array before running out of memory, compared to the other sorting algorithms.

2 Experimental Setup and Procedure

This experiment will be conducted through the Java Programming Language, where the code will be written and executed in the Eclipse IDE for Java Developers. The implemented code is written in Java 22, and utilizes 3 built-in classes from the `java.util` package. Namely, `ArrayList`, `Collections`, and `Random`.

The script responsible for generating the input data to be tested upon is written by the author themselves, utilizing the `Random` class to generate the random numbers, and `ArrayList` to store. In some cases, the `Collections` class was also used to conveniently sort the generated numbers for the special inputs where the array is already sorted in ascending and descending orders. To ensure the array is deterministic so that for each run of the sorting algorithms operate on the same array, a seed equal to the length of the array was supplied. This way, the generated arrays all have the same elements to ensure no sorting algorithms got an easier array to work with, leading to faster runtime than average.

Generating the array itself involved a `for` loop to generate a random number, then appending it to an initialized `ArrayList` object that would be outputted as the resulting random integer array.

The sorting algorithms implemented for testing have all been adapted from the code snippets found on Wikipedia. Links to these algorithms can be found in the documentation of the functions within the source code itself.

To measure the execution runtime of the algorithms, the function `executionTime` records the program runtime before and after the sorting algorithm is executed. Then, the algorithm's execution time is calculated by taking the difference between the end and start time. This runtime is then printed along with the length of the array divided by 1000. This is run 20 times in a `for` loop, and the resulting length and algorithm runtime is recorded in a 2D graph as shown in the results section below.

In relation to measuring the memory usage, a binary search approach was implemented to find an approximate maximum array size. Since either a process can crash due to an out-of-memory error or not, and there is a certain cutoff point between when that happens, the binary search method is applicable for solving this problem. A two-pointers approach was taken along with a `while` loop to test the midpoint length of the two pointers' array length. To test if an out-of-memory exception would occur, a `try-catch` loop was implemented

to catch these exceptions and limit the maximum array length to that of the midpoint minus one. Otherwise, the program would limit the lower bound instead to the midpoint plus one.

This loop will repeat until the lower pointer exceeds the higher pointer, in which case it will return the last array length that didn't result in a program crash, which would be our maximum array length. This maximized result is then recorded on a bar graph, to be compared with the maximum array length that the other sorting algorithms require as well. This can be seen in the following section too.

The source code utilized for this experiment can be found in the ZIP file where this report file is located. The source code has been given brief documentations on the details of the algorithms themselves, as well as any further explanation for measuring the execution time and the maximum array size.

3 Results

3.1 Time Performance of Random Integers

The first aim of the experiment is to compare the runtime performance of each of the sorting algorithms. The comparison will be categorized based on the input array, one consisting of thousands of integers and the other comprising tens of integers.

Large Array Input

This section examines the performance when the array contains thousands of integers. Figure 2 shows the resulting runtime of the 6 sorting algorithms. From the figure, it is apparent that Bubble Sort performed the worst, taking almost triple the time to sort large amounts of integers compared to the second worst. Moreover, it can also be seen that Merge Sort, Quick Sort, and Heap Sort all performed nearly instantaneously, taking no more than 30 milliseconds to sort thousands of integers.

Small Array Input

In contrast to the previous section, this paragraph will focus on small sized arrays. Specifically, arrays of size $5n$ where $0 < n \leq 5$. Figure 3 provides the data for the execution time of the sorting algorithms. From the figure, it is apparent that no definitive trend emerged from any of the sorting algorithms when working with arrays in a small size. Although there are still differences in performance, the overall benefit in speed is inconsequential when measuring in nanoseconds.

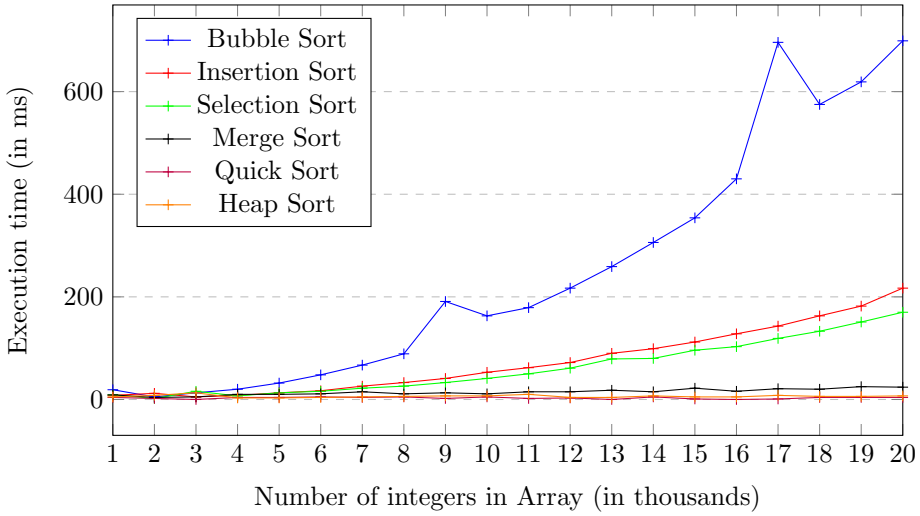

Figure 2: Execution time of the sorting algorithms with large amounts of random numbers

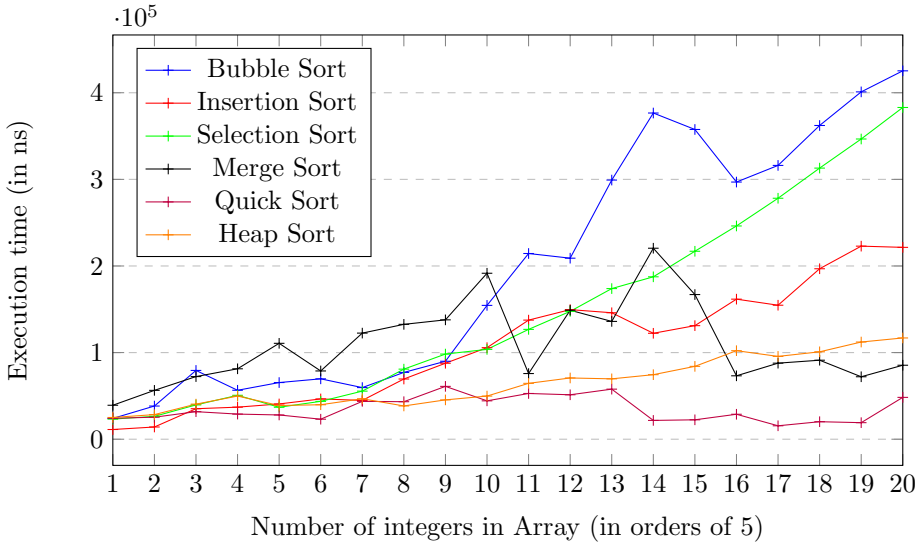

Figure 3: Execution time of the sorting algorithms with small amounts of random numbers

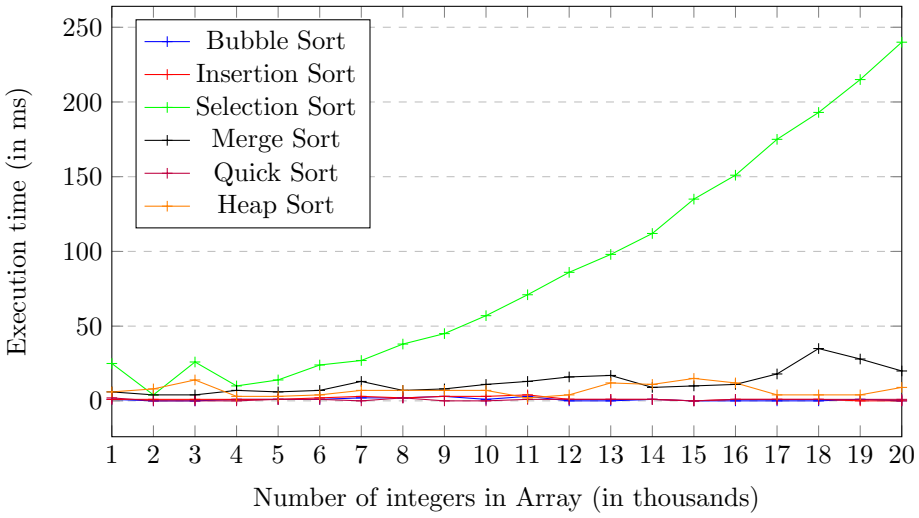

Figure 4: Execution time of the sorting algorithms with random ascending numbers

3.2 Time Performance of Certain Inputs

The next research question to address concerns the runtime performance of different sorting algorithms working on some special inputs. This subsection has been differentiated into 3 segments, each focusing on a special type of random numbers input.

Random Ascending Integers

This part will explore the execution time of sorting algorithms when the input is already sorted in ascending order. Figure 4 presents the runtime performance of the 6 sorting algorithms. Interestingly, the only algorithm to do worse as the input grew later was Selection Sort, whereas the other five algorithms all did no worse than 50 milliseconds. Another expected observation to be made is that when the input is already sorted accordingly, both Bubble Sort and Insertion Sort outperformed the fastest algorithms in random integers input.

Random Descending Integers

Turning to the opposite of the previous special input, this section now considers a list of numbers sorted in descending order. Figure 5 compares the different sorting algorithms runtime when dealing with random decreasing numbers. It can be seen in the figure that Bubble Sort performed faster than Selection Sort for most of the inputs, while Merge Sort, Quick Sort, and Heap Sort continues to be vastly faster than the other sorting algorithms.

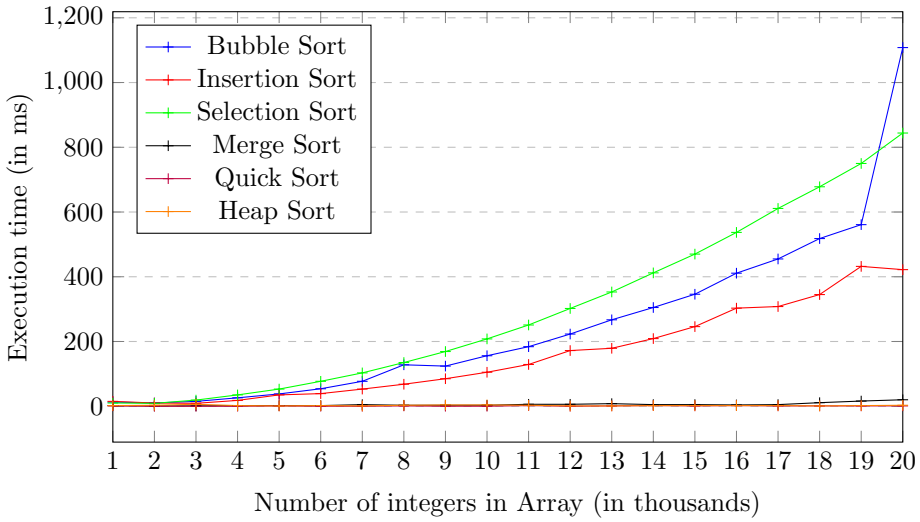

Figure 5: Execution time of the sorting algorithms with random descending numbers

Random Restricted Integers

Finally, this section focuses on the input where the numbers are bounded, ensuring the occurrence duplicates integers in the list. Figure 6 displays the runtime of these algorithms working with the inputs. Despite the figure looking very similar to some other inputs, the runtime for each algorithm have been reduced significantly. However, Bubble Sort still performs the worst out of the 4 algorithms, having its runtime at large inputs nearly quadrupled that of the second worst algorithm, Insertion Sort.

3.3 Memory Usage

To answer the third experiment aim, we will utilize the algorithm as detailed in the Experimental Setup and Procedure section. Table 7 shows the maximum array size given the maximum heap size allocated for the Java Virtual Machine is four megabytes. From the graph, it can be seen that Merge Sort used the most memory, only just managing an array of 40 thousand. Meanwhile, Bubble Sort, Selection Sort, and Heap Sort used the least amount of memory, sorting arrays at the maximum size of 130 thousand.

4 Discussion

After presenting the results, this section will reflect on the findings and come up with plausible explanations for why it does or does not match with the hypothesis. This section will discuss the performance of algorithms with respect

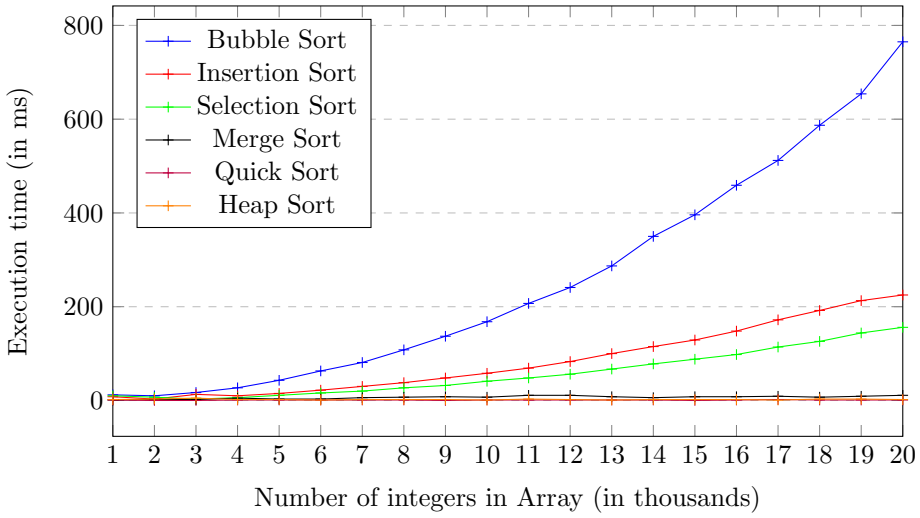

Figure 6: Execution time of the sorting algorithms with random numbers between 1 (inclusive) and 100 (exclusive)

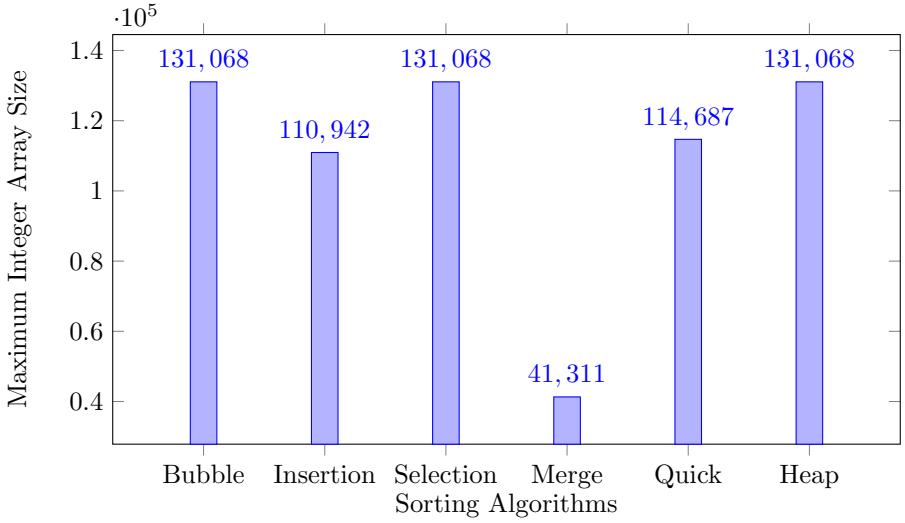

Figure 7: Maximum array length for each sorting algorithm with maximum heap size of 4 megabytes

to time under random integer arrays of small and big sizes, time under special random integer arrays, and the memory usage of each algorithm.

4.1 Execution Time of Random Integer Arrays of Varied Size

As expected from Figure 2, Bubble Sort, Insertion Sort, and Selection Sort all have the same growth rate, albeit some grew faster than others. This lines up with our hypothesis, as the time complexity of those 3 algorithms grows quicker than the time complexity of Merge Sort, Quick Sort, and Heap Sort in the average-case. Although, when dealing with smaller inputs, the results tend to vary themselves in a more chaotic manner than the bigger inputs, as can be seen in Figure 3. This is also expected, as asymptotic notation only bounds after a large enough input, as given by the formal definitions in the introduction section. Similar results were also found in Karunanithi's own investigation into the performance of sorting algorithms [3].

4.2 Execution Time of Special Random Integer Arrays

The purpose of this section is to look at the performance when given the best and worst-case inputs. Observing the graph in Figure 4, it is clear that there were massive improvements for Bubble Sort and Insertion Sort. Indeed, Bubble Sort would only swap if there still are elements out-of-order, which is not the case with an already sorted array of numbers. Similarly, Insertion Sort checks from the last sorted array index, which is always smaller than the current index its comparing to in a sorted array. Hence, the insanely fast speeds that Bubble Sort and Insertion Sort reached, which is $O(n)$ as only one pass of the array was needed.

Surprisingly, while Quick Sort was hypothesized to perform the worst when the array is already sorted in either ascending or descending order, it still outperformed every other algorithm no matter the input. This is caused by the implementation of Quick Sort, in that the chosen pivot was not the first nor last element, but the middle element, as specified for the implementation of "Fat Partitioning". This led to the list division being a perfect even split, thus contributing no extra time.

When confronted with integer arrays with multiple repeated elements, the performance showed little to no difference to the one with random integer arrays. This suggests that the element values itself has no importance when it comes to sorting, other than comparison. However, this experiment is treating all integers are the same, that is, an integer is indistinguishable from another integer with the same value. If one wishes to sort an array and keep the same order if duplicated values arise, then certain algorithms will not be applicable for these kinds of tasks. This is not a focus on the experiment, however it is important to notice when taking into consideration what sorting algorithms to implement for a project.

4.3 Memory Usage of Sorting Algorithms

Finally, exploring the topic of memory usage, this experiment also set out to compute the maximum length of an array before an out of memory exception was thrown. Insertion Sort and Quick Sort returned slightly worse array lengths than the expected, possibly due to the amount of variables, miscellaneous helper functions, or function calls for code organization. However, Merge Sort performed the worst out of the 6 algorithms, which is what was expected, considering its memory complexity was the only one in linear usage $O(n)$. Indeed, Merge Sort is not an in-place algorithm, meaning it creates a new list with the sorted items rather than modifying its input array, hence the lower maximum array length.

Identical results can also be seen from Karunanithi's investigation too, where the findings he came up with showed that every sorting algorithm used the same amount of memory space, other than Merge Sort, which had close to double the memory consumption than the rest [3].

5 Conclusion

This report has explored the performance of 6 sorting algorithms in a variety of different inputs. The time and memory performance of which was recorded in graphs for meaningful discussions and interpretations of the data. Furthermore, it was also discussed the common methods of measuring algorithmic efficiency with the asymptotic notation and worst-case analysis.

Although this report has tried to be as comprehensive as possible, there are still areas where this report failed at. The maximum input array experiment had to have limited maximum memory heap of four megabytes for the Java Virtual Machine, as the default, which is less than one-fourth of the total available physical memory [5], would take too long to compute for each search iteration. For future research opportunities, a different approach to calculating the memory usage can be taken, although it could be considered a challenge due to how non-deterministic memory allocation on a machine can be.

This report is beneficial for prospective computer science students to introduce them to the world of algorithmic thinking and optimizations. Sorting algorithms are simple procedures, yet have incredible depth as to what approaches can be taken to solve the problem, as well as techniques to analyze theoretical performance of an algorithm. Through this report, the hope is that an understanding regarding computational complexity can be reached.

Bibliography

[1] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. 4th printing 2016. New York: Cambridge University Press, 2016. 579 pp. ISBN: 978-0-521-42426-4.

[2] Thomas H. Cormen et al. *Introduction to Algorithms*. Fourth edition. Cambridge, Massachusett: The MIT Press, 2022. 1291 pp. ISBN: 978-0-262-04630-5.

[3] Ashok Kumar Karunanithi et al. "A Survey, Discussion and Comparison of Sorting Algorithms". In: *Department of Computing Science, Umea University* (2014).

[4] Steven S. Skiena. *The Algorithm Design Manual*. 2. ed., [Nachdr.] London: Springer, 2010. 730 pp. ISBN: 978-1-84996-720-4 978-1-84800-069-8.

[5] *The Parallel Collector*. URL: `https://docs.oracle.com/javase/8/docs/technotes/guides/vm/gctuning/parallel.html#CHDCFBIF` (visited on 03/15/2025).

[6] Salil Vadhan. "Computational Complexity". In: *Encyclopedia of Cryptography and Security*. Ed. by Henk C. A. Van Tilborg and Sushil Jajodia. Boston, MA: Springer US, 2011, pp. 235–240. ISBN: 978-1-4419-5905-8 978-1-4419-5906-5. DOI: `10.1007/978-1-4419-5906-5_442`. URL: `http://link.springer.com/10.1007/978-1-4419-5906-5_442` (visited on 03/15/2025).